Sesión 1
INTRODUCCIÓN AL MACHINE LEARNING

Panorama del Curso y Fundamentos

Por
Carlos César Sánchez Coronel

2026

Índice general

Índice general **1**

1. Introducción al Machine Learning y Panorama del Curso **2**
1.1. Logro de la sesión 2
1.2. Definición de Machine Learning e Inteligencia Artificial 2
1.3. Tipos de aprendizaje 3
1.3.1. Aprendizaje supervisado 3
1.3.2. Aprendizaje no supervisado 3
1.3.3. Aprendizaje por refuerzo 3
1.3.4. Aprendizaje semi-supervisado y autosupervisado 3
1.4. Modelos de Machine Learning: fundamentos matemáticos y computacionales 3
1.4.1. Modelos supervisados 4
1.4.2. Modelos no supervisados 8
1.4.3. Aprendizaje por refuerzo (conceptos básicos) 9
1.5. Ciclo de vida de un proyecto de Machine Learning: Metodología CRISP-DM 10
1.6. Proyecto End-to-End ilustrativo: Predicción de precios de viviendas 11
1.7. Importancia de la matemática en Machine Learning 11

Capítulo 1

Introducción al Machine Learning y Panorama del Curso

En esta primera sesión se establecen las bases conceptuales del machine learning, se exploran los diferentes tipos de aprendizaje, se presentan los modelos fundamentales con sus fundamentos matemáticos y computacionales, y se describe el ciclo de vida de un proyecto de machine learning siguiendo metodologías estándar como CRISP-DM. El objetivo es que el estudiante comprenda el panorama global de la disciplina y se sitúe para los temas avanzados que se abordarán en el curso.

1.1. Logro de la sesión

Al finalizar la sesión, el estudiante será capaz de:

- Comprender el flujo completo de un proyecto de machine learning.
- Identificar los tipos de aprendizaje y los modelos más representativos.
- Reconocer los fundamentos matemáticos y computacionales que subyacen a cada modelo.
- Situar el resto del curso en un contexto global.

1.2. Definición de Machine Learning e Inteligencia Artificial

La **Inteligencia Artificial (IA)** es un campo amplio que busca crear sistemas capaces de realizar tareas que requieren inteligencia humana, como el razonamiento, la percepción, el aprendizaje y la toma de decisiones. El **Machine Learning (ML)** es un subcampo de la IA que se centra en desarrollar algoritmos que permiten a las máquinas aprender a partir de datos, sin ser programadas explícitamente para cada tarea. Dentro del ML, el **Deep Learning** utiliza redes neuronales profundas para modelar patrones complejos, especialmente en dominios como visión por computador y procesamiento del lenguaje natural.

1.3. Tipos de aprendizaje

Los algoritmos de machine learning se clasifican según la naturaleza de la experiencia de la que disponen para aprender.

1.3.1. Aprendizaje supervisado

Se dispone de un conjunto de datos etiquetados, es decir, para cada entrada se conoce la salida deseada. El objetivo es aprender una función que mapee entradas a salidas.

- **Regresión:** La salida es un valor continuo (ej. precio de una vivienda, temperatura).
- **Clasificación:** La salida es una categoría discreta (ej. spam / no spam, tipo de flor).

1.3.2. Aprendizaje no supervisado

No hay etiquetas. El algoritmo debe encontrar estructuras ocultas en los datos.

- **Clustering:** Agrupar datos similares (ej. segmentación de clientes).
- **Reducción de dimensionalidad:** Representar los datos con menos variables (ej. visualización, compresión).
- **Asociación:** Encontrar reglas que relacionen ítems (ej. "clientes que compran pan también compran leche").

1.3.3. Aprendizaje por refuerzo

Un agente aprende a tomar decisiones interactuando con un entorno. Recibe recompensas (o castigos) por sus acciones y debe maximizar la recompensa acumulada. Es el paradigma utilizado en robótica, juegos y vehículos autónomos.

1.3.4. Aprendizaje semi-supervisado y autosupervisado

Combinan datos etiquetados y no etiquetados. El aprendizaje autosupervisado es especialmente relevante en NLP, donde se generan etiquetas a partir de los propios datos (ej. predecir la siguiente palabra en una frase).

1.4. Modelos de Machine Learning: fundamentos matemáticos y computacionales

A continuación se describen los modelos más representativos, con énfasis en su formulación matemática, implementación computacional y aplicaciones reales.

1.4.1. Modelos supervisados

Regresión lineal

- **Definición y uso:** Modelo que asume una relación lineal entre las variables de entrada y la salida continua. Es uno de los modelos más simples y ampliamente utilizados como punto de partida en problemas de regresión.

- **Aplicaciones reales:** Predicción de precios de viviendas, estimación de ventas, forecasting de series temporales, análisis de tendencias.

- **Fundamento matemático:** Dado un conjunto de N muestras $\{(x_i, y_i)\}_{i=1}^N$ con $x_i \in \mathbb{R}^n$, se busca un vector de pesos $\mathbf{w} \in \mathbb{R}^n$ y un sesgo b (a menudo incluido en $\mathbf{w}$ aumentando x con un 1) tales que:

  $$y_i \approx \mathbf{w}^T x_i + b.$$

  La función de coste más común es el error cuadrático medio:

  $$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \mathbf{w}^T x_i)^2.$$

  La solución analítica (ecuación normal) es:

  $$\mathbf{w} = (X^T X)^{-1} X^T \mathbf{y},$$

  donde X es la matriz de diseño $(N \times n)$ y $\mathbf{y}$ el vector de salidas.

- **Fundamento computacional:** La ecuación normal requiere invertir una matriz, cuyo coste es $O(n^3)$. Para grandes volúmenes de datos o muchas características, se prefiere el gradiente descendente, que actualiza los pesos iterativamente:

  $$\mathbf{w} := \mathbf{w} - \alpha \nabla J(\mathbf{w}),$$

  siendo α la tasa de aprendizaje. El gradiente para la regresión lineal es:

  $$\nabla J(\mathbf{w}) = \frac{2}{N} X^T (X\mathbf{w} - \mathbf{y}).$$

- **Casos de uso:** Modelos baseline, interpretabilidad, relaciones lineales aproximadas.

Regresión logística

- **Definición y uso:** A pesar de su nombre, es un modelo de clasificación binaria. Modela la probabilidad de pertenencia a una clase.

- **Aplicaciones reales:** Detección de spam, diagnóstico médico (enfermedad sí/no), predicción de abandono de clientes, análisis de riesgo crediticio.

- **Fundamento matemático:** Se asume que la probabilidad de que una muestra pertenezca a la clase 1 viene dada por la función sigmoide:

  $$P(y = 1|x) = \sigma(\mathbf{w}^T x) = \frac{1}{1 + e^{-\mathbf{w}^T x}}.$$

Para estimar los pesos se maximiza la log-verosimilitud (o se minimiza la log-loss):

$$\mathcal{L}(\mathbf{w}) = -\sum_{i=1}^{N} \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right],$$

donde $\hat{y}_i = \sigma(\mathbf{w}^T x_i)$. No tiene solución cerrada; se optimiza mediante gradiente descendente.

- **Fundamento computacional:** El gradiente de la log-loss es:

$$\nabla \mathcal{L}(\mathbf{w}) = \sum_{i=1}^{N} (\hat{y}_i - y_i) x_i.$$

  La implementación eficiente utiliza lotes (mini-batch) y regularización (L1, L2) para evitar sobreajuste.

- **Casos de uso:** Problemas de clasificación binaria donde se requiere probabilidad de pertenencia.

k-Vecinos más cercanos (k-NN)

- **Definición y uso:** Método no paramétrico que clasifica o predice basándose en la distancia a los k ejemplos más cercanos del conjunto de entrenamiento.

- **Aplicaciones reales:** Sistemas de recomendación, reconocimiento de patrones, motores de búsqueda de imágenes similares.

- **Fundamento matemático:** Dado un punto de consulta x_q, se encuentran los k puntos más cercanos según una métrica de distancia, típicamente la distancia euclidiana:

$$d(x_q, x_i) = \sqrt{\sum_{j=1}^{n} (x_q^j - x_i^j)^2}.$$

  Para clasificación, la etiqueta se asigna por voto mayoritario; para regresión, se promedian los valores de salida de los vecinos.

- **Fundamento computacional:** La búsqueda de vecinos puede ser costosa ($O(N)$ por consulta). Se utilizan estructuras como KD-Trees o Ball Trees para acelerar, especialmente en bajas dimensiones. Para datos de alta dimensionalidad, la distancia euclidiana pierde efectividad (maldición de la dimensionalidad).

- **Casos de uso:** Datos con pocas dimensiones, prototipado rápido, problemas donde la relación local es importante.

Naive Bayes

- **Definición y uso:** Clasificador probabilístico basado en el teorema de Bayes con la suposición de independencia condicional entre las características.

- **Aplicaciones reales:** Clasificación de textos (filtro de spam), análisis de sentimiento, diagnóstico médico.

- **Fundamento matemático:** Dada una clase C_k, la probabilidad posterior es:

$$P(C_k|x) = \frac{P(C_k)\prod_{j=1}^{n} P(x_j|C_k)}{P(x)}.$$

La clasificación se realiza eligiendo la clase que maximiza $P(C_k)\prod_j P(x_j|C_k)$. Para características continuas, se suele asumir una distribución normal:

$$P(x_j|C_k) = \frac{1}{\sqrt{2\pi\sigma_{kj}^2}} \exp\left(-\frac{(x_j-\mu_{kj})^2}{2\sigma_{kj}^2}\right).$$

- **Fundamento computacional:** El entrenamiento consiste en estimar las probabilidades a priori y los parámetros de las distribuciones (frecuencias para discretas, medias y varianzas para continuas). Es muy rápido y escalable a grandes conjuntos de datos. La predicción también es veloz.

- **Casos de uso:** Problemas con muchas características, especialmente texto; cuando la independencia condicional es aproximadamente válida.

Máquinas de vectores de soporte (SVM)

- **Definición y uso:** Modelo que busca el hiperplano que maximiza el margen entre clases. Puede extenderse a problemas no lineales mediante el truco del kernel.

- **Aplicaciones reales:** Clasificación de imágenes, bioinformática, detección de fraudes, reconocimiento de escritura.

- **Fundamento matemático:** Para un problema linealmente separable, se busca un hiperplano $\mathbf{w}^T x + b = 0$ que maximice la distancia a los puntos más cercanos (vectores soporte). Esto conduce al siguiente problema de optimización convexa:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{sujeto a} \quad y_i(\mathbf{w}^T x_i + b) \geq 1, \ \forall i.$$

Para datos no separables, se introducen variables de holgura y un parámetro de regularización C. El truco del kernel permite trabajar en espacios de características de alta dimensión sin calcular explícitamente la transformación:

$$f(x) = \sum_{i=1}^{N} \alpha_i y_i K(x_i, x) + b,$$

donde K es una función kernel (ej. lineal, polinómico, RBF).

- **Fundamento computacional:** La resolución del problema dual implica una matriz de tamaño $N \times N$, costosa para grandes N. Se usan algoritmos de optimización específicos (SMO). Los kernels permiten modelar relaciones no lineales, pero requieren ajuste de hiperparámetros.

- **Casos de uso:** Problemas de clasificación con dimensiones moderadas, cuando se requiere un margen amplio y buena generalización.

Árboles de decisión

- **Definición y uso:** Modelo que particiona el espacio de características mediante reglas binarias, formando una estructura de árbol. Fácil de interpretar.
- **Aplicaciones reales:** Diagnóstico médico, análisis de riesgo crediticio, sistemas de soporte a la decisión.
- **Fundamento matemático:** La partición se realiza seleccionando en cada nodo la característica y el umbral que maximizan la pureza de los hijos. Para clasificación, se usan medidas como el índice de Gini:

  $$G = 1 - \sum_{k=1}^{K} p_k^2,$$

  o la entropía:

  $$H = -\sum_{k=1}^{K} p_k \log p_k,$$

  donde p_k es la proporción de muestras de la clase k en el nodo. Para regresión, se minimiza el error cuadrático medio.
- **Fundamento computacional:** El entrenamiento tiene coste $O(N \cdot n \cdot \log N)$ si se ordenan las características. Los árboles tienden a sobreajustar, por lo que se podan o limitan (profundidad máxima, mínimo de muestras por hoja).
- **Casos de uso:** Problemas donde la interpretabilidad es crucial, detección de interacciones entre variables.

Random Forest

- **Definición y uso:** Conjunto de muchos árboles de decisión entrenados con diferentes subconjuntos de datos (bootstrap) y características (random subspace). La predicción final es el promedio (regresión) o voto mayoritario (clasificación).
- **Aplicaciones reales:** Predicción de demanda, selección de características, problemas de clasificación y regresión en general.
- **Fundamento matemático:** Cada árbol se entrena con una muestra bootstrap del conjunto original. En cada división, solo se considera un subconjunto aleatorio de características. Esto reduce la correlación entre árboles y, en conjunto, disminuye la varianza sin aumentar el sesgo. La predicción final es:

  $$\hat{y} = \frac{1}{T}\sum_{t=1}^{T} \hat{y}_t \quad \text{(regresión)}, \quad \hat{y} = \text{moda}\{\hat{y}_1, \ldots, \hat{y}_T\} \quad \text{(clasificación)}.$$

- **Fundamento computacional:** El entrenamiento es paralelizable (cada árbol independiente). La predicción también lo es. Es robusto al sobreajuste y maneja bien datos de alta dimensionalidad.
- **Casos de uso:** Modelo por defecto en muchos problemas tabulares, importancia de variables.

Redes neuronales (Perceptrón multicapa - MLP)

- **Definición y uso:** Modelo compuesto por capas de neuronas interconectadas, cada neurona realiza una combinación lineal seguida de una función de activación no lineal. Son la base del deep learning.
- **Aplicaciones reales:** Visión por computador, procesamiento de lenguaje natural, reconocimiento de voz, sistemas de recomendación, juegos.
- **Fundamento matemático:** Una neurona computa:

$$a = \sigma\left(\sum_{j=1}^{n} w_j x_j + b\right),$$

  donde σ es una función de activación (ReLU, sigmoide, tanh). La salida de la red se obtiene componiendo sucesivas capas. El aprendizaje se realiza minimizando una función de pérdida (ej. error cuadrático, entropía cruzada) mediante retropropagación (backpropagation), que calcula gradientes usando la regla de la cadena.
- **Fundamento computacional:** Las operaciones se expresan como multiplicaciones de matrices y aplicaciones elemento a elemento, lo que permite una implementación eficiente en GPUs. Los frameworks modernos (TensorFlow, PyTorch) automatizan el cálculo de gradientes y la optimización.
- **Casos de uso:** Prácticamente cualquier tarea, especialmente cuando se dispone de grandes volúmenes de datos y potencia de cómputo.

1.4.2. Modelos no supervisados

K-Means

- **Definición y uso:** Algoritmo de clustering que particiona los datos en k grupos, donde cada punto pertenece al cluster con el centroide más cercano.
- **Aplicaciones reales:** Segmentación de clientes, compresión de imágenes, agrupamiento de documentos.
- **Fundamento matemático:** Se busca minimizar la suma de distancias cuadráticas intra-cluster:

$$J = \sum_{j=1}^{k} \sum_{x \in C_j} \|x - \mu_j\|^2,$$

  donde μ_j es el centroide del cluster C_j. El algoritmo itera entre dos pasos:

  1. Asignar cada punto al centroide más cercano.
  2. Recalcular cada centroide como la media de los puntos asignados.
- **Fundamento computacional:** Cada iteración cuesta $O(N \cdot k \cdot n)$. Es sensible a la inicialización; se suele ejecutar varias veces con diferentes semillas y elegir la mejor. Converge a un óptimo local.
- **Casos de uso:** Datos con clusters aproximadamente esféricos y de tamaño similar.

PCA (Análisis de Componentes Principales)

- **Definición y uso:** Técnica de reducción de dimensionalidad que proyecta los datos a un nuevo espacio de menor dimensión, conservando la máxima varianza.
- **Aplicaciones reales:** Visualización de datos, preprocesamiento, reducción de ruido, compresión.
- **Fundamento matemático:** Dada la matriz de datos centrada X $(N \times n)$, se calcula la matriz de covarianza:
$$\Sigma = \frac{1}{N-1} X^T X.$$
Los autovectores de Σ (ordenados por autovalores decrecientes) son las direcciones de máxima varianza. La proyección a d dimensiones es:
$$X_{\text{pca}} = XW_d,$$
donde W_d contiene los primeros d autovectores.
- **Fundamento computacional:** La descomposición espectral tiene coste $O(n^3)$, pero se puede usar SVD (descomposición en valores singulares) para mayor estabilidad numérica y eficiencia. Para matrices grandes, existen métodos iterativos (ej. power iteration).
- **Casos de uso:** Datos con muchas variables correlacionadas, antes de aplicar otros modelos.

DBSCAN

- **Definición y uso:** Algoritmo de clustering basado en densidad que agrupa puntos que están estrechamente empaquetados, marcando como ruido los puntos aislados.
- **Aplicaciones reales:** Detección de anomalías, datos con formas arbitrarias, análisis espacial.
- **Fundamento matemático:** Define un punto como núcleo si tiene al menos $minPts$ vecinos a distancia ϵ. Los puntos alcanzables desde un núcleo forman un cluster. Formalmente, se basa en la relación de vecindad y conectividad por densidad.
- **Fundamento computacional:** La búsqueda de vecinos puede ser costosa ($O(N^2)$) si no se usan estructuras de índices espaciales. Es sensible a los parámetros ϵ y $minPts$.
- **Casos de uso:** Datos con clusters de formas complejas, presencia de ruido.

1.4.3. Aprendizaje por refuerzo (conceptos básicos)

- **Definición y uso:** Un agente aprende una política de comportamiento mediante la interacción con un entorno, recibiendo recompensas.
- **Aplicaciones reales:** Robótica, juegos (AlphaGo, Atari), vehículos autónomos, optimización de recursos.

- **Fundamento matemático:** Se modela como un Proceso de Decisión de Markov (MDP) definido por (S, A, P, R, γ). El objetivo es maximizar la recompensa acumulada descontada:
$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$
La función de valor $V(s)$ estima el retorno esperado desde el estado s, y la función de valor de acción $Q(s, a)$ estima el retorno esperado al tomar la acción a en el estado s. La ecuación de Bellman relaciona valores sucesivos:
$$V(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s') \right).$$
El algoritmo Q-learning actualiza la tabla Q mediante:
$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$
- **Fundamento computacional:** Para espacios grandes, se usan aproximaciones con redes neuronales (Deep Q-Networks). El equilibrio entre exploración y explotación es clave (ej. ϵ-greedy).
- **Casos de uso:** Problemas secuenciales con recompensas retardadas.

1.5. Ciclo de vida de un proyecto de Machine Learning: Metodología CRISP-DM

El desarrollo de un proyecto de ML no se limita a entrenar un modelo; sigue un proceso iterativo que garantiza que la solución cumpla los objetivos de negocio. La metodología más utilizada es CRISP-DM (Cross-Industry Standard Process for Data Mining), que consta de las siguientes fases:

1. **Comprensión del negocio:** Definir objetivos y requisitos desde una perspectiva de negocio. Traducir el problema a una tarea de ML.
2. **Comprensión de los datos:** Recolectar datos iniciales, explorarlos, verificar calidad, detectar patrones interesantes.
3. **Preparación de los datos:** Seleccionar, limpiar, transformar y enriquecer los datos para el modelado. Incluye manejo de nulos, codificación de variables, normalización, etc.
4. **Modelado:** Seleccionar y aplicar técnicas de modelado, ajustar hiperparámetros. A menudo se itera con la fase anterior.
5. **Evaluación:** Evaluar el modelo con métricas adecuadas y validar que cumple los objetivos de negocio. Se puede revisar si el modelo es suficientemente bueno o si se necesita mejorar.
6. **Despliegue:** Poner el modelo en producción, integrarlo con sistemas existentes, monitorear su rendimiento y actualizarlo cuando sea necesario.

Otras metodologías como KDD o TDSP comparten principios similares, pero CRISP-DM sigue siendo el estándar de facto.

1.6. Proyecto End-to-End ilustrativo: Predicción de precios de viviendas

Para fijar ideas, se describe cómo se aplicaría CRISP-DM a un caso concreto: predecir el precio de venta de viviendas en una ciudad.

1. **Comprensión del negocio:** Una inmobiliaria desea estimar precios de manera automática para fijar precios competitivos y asesorar a clientes.
2. **Comprensión de los datos:** Se recolectan datos históricos de ventas: características como metros cuadrados, número de habitaciones, ubicación, antigüedad, etc. Se realiza un análisis exploratorio (EDA) para visualizar distribuciones, correlaciones y detectar outliers.
3. **Preparación de los datos:** Se imputan valores nulos (por ejemplo, con la media), se normalizan las variables numéricas, se codifica la ubicación (one-hot encoding), se divide en entrenamiento y prueba.
4. **Modelado:** Se prueban varios modelos: regresión lineal, árbol de decisión, random forest. Se ajustan hiperparámetros mediante validación cruzada.
5. **Evaluación:** Se comparan las métricas (RMSE, MAE, R^2) en el conjunto de prueba. Se analiza si el error es aceptable para el negocio. Se interpretan las variables más importantes (por ejemplo, la ubicación es clave).
6. **Despliegue:** Se desarrolla una API REST con el modelo entrenado, se integra en la web de la inmobiliaria y se monitorea el rendimiento con nuevos datos.

Este flujo ilustra cómo los conceptos de la sesión se integran en un proyecto real.

1.7. Importancia de la matemática en Machine Learning

Para comprender en profundidad los modelos y no limitarse a un uso meramente instrumental, es fundamental dominar ciertas áreas de la matemática:

- **Álgebra lineal:** Vectores, matrices, autovalores, descomposiciones. Es la base de las representaciones y transformaciones lineales.
- **Cálculo:** Derivadas, gradientes, optimización. Necesario para entrenar modelos mediante gradiente descendente y backpropagation.
- **Probabilidad y estadística:** Distribuciones, inferencia, estimación, test de hipótesis. Fundamental para evaluar modelos y entender la incertidumbre.

En las siguientes sesiones del curso se profundizará en estos conceptos aplicados a cada modelo, con el fin de construir una base sólida que permita al estudiante no solo aplicar, sino también innovar y adaptar técnicas de machine learning.